# Documentação MecaniDoc

**Projeto:** MecaniDoc — aplicação Next.js com Supabase, checkout e Stripe.
**Gerado:** documentação consolidada a partir dos ficheiros do repositório.

---

## Índice


1. Visão geral e arranque (README)
2. Variáveis de ambiente (.env)
3. Configuração Stripe
4. Build sem Stripe / correções de build
5. Tabela `order_items` pelo Dashboard Supabase
6. SUPABASE_DB_URL e checkout ( `order_items` )
7. Script SQL: corrigir API `order_items`
8. Scripts SQL no repositório (referência)
9. Scripts npm


---

## 1. Visão geral e arranque (README)

Este é um projeto Next.js criado com `create-next-app` .

### Arranque

Execute o servidor de desenvolvimento:

```
npm run dev
# ou
yarn dev
# ou
pnpm dev
# ou
bun dev
```

Abra http://localhost:3000 no browser.

O projeto usa `next/font` para otimizar e carregar a família de fontes Geist.

### Recursos Next.js

- Documentação Next.js
- Learn Next.js
- Repositório Next.js no GitHub

### Deploy

O deploy recomendado é na Vercel. Ver documentação de deploy Next.js.

---

## 2. Variáveis de ambiente (.env)

### Para o Next.js ler o `.env`

1. **Ficheiro no sítio certo**
   O ficheiro deve chamar-se `.env` ou `.env.local` e ficar na **raiz do projeto** (mesma pasta que `package.json`).

2. **Nomes exatos (Supabase)**

   - `NEXT_PUBLIC_SUPABASE_URL` — URL do projeto (ex.: `https://xxxx.supabase.co`)
   - `NEXT_PUBLIC_SUPABASE_ANON_KEY` — chave "anon" / pública (começa por `eyJ...`)

   Sem o prefixo `NEXT_PUBLIC_` estas variáveis **não** ficam disponíveis no browser e o login pode falhar.

3. **Reiniciar o servidor**
   Depois de alterar o `.env` ou `.env.local`, pare o `npm run dev` (Ctrl+C) e volte a executar:

   ```
   npm run dev
   ```

4. **Sem espaços ou aspas a mais**
   No `.env` use uma linha por variável, sem espaços à volta do `=` e sem aspas à volta do valor (a não ser que o valor tenha espaços):

   ```
   NEXT_PUBLIC_SUPABASE_URL=https://yjdoprfjgaeyvvyhpihd.supabase.co
   NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
   ```

### Se o login der 400 (Bad Request)

- Confirme no **Supabase Dashboard** > **Authentication** > **Users** que o utilizador existe e que o email está **confirmado**.
- Confirme que a **anon key** no `.env` é a do mesmo projeto que a URL (Dashboard > Project Settings > API).
- Teste o login com a mesma senha que definiu no Supabase (ou use "Reset password" no Dashboard).

---

## 3. Configuração Stripe

### Instalação

```
npm install stripe @stripe/stripe-js
```

### Variáveis de ambiente

Adicione ao `.env`:

```
# Stripe
STRIPE_SECRET_KEY=sk_test_your_stripe_secret_key
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key
STRIPE_WEBHOOK_SECRET=whsec_your_webhook_secret

# Supabase Service Role (necessário para webhooks)
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key
```

### Obter chaves Stripe

1. Dashboard Stripe → **Developers** > **API keys**
2. Copie **Secret key** ( `sk_test_` ou `sk_live_` )
3. Copie **Publishable key** ( `pk_test_` ou `pk_live_` )

### Webhook

1. **Developers** > **Webhooks** → **Add endpoint**
2. URL: `https://seu-dominio.com/api/stripe/webhook`
3. Eventos recomendados:
   - **`checkout.session.completed`** (obrigatório para Stripe Checkout)
   - `payment_intent.succeeded`
   - `payment_intent.payment_failed`
   - `charge.refunded`
4. Copie o **Signing secret** ( `whsec_` ) → `STRIPE_WEBHOOK_SECRET` no `.env`

### Script SQL

Execute `add_stripe_fields.sql` no Supabase para campos necessários na tabela `orders`.

### Integração no código

**API routes:**

- `/api/stripe/create-payment-intent` — cria PaymentIntent
- `/api/stripe/webhook` — processa webhooks
- `/api/sales` — consulta vendas (utilizadores master)

**Páginas:**

- `/dashboard/admin/sales` — consulta de vendas

**Componentes:**

- `src/app/checkout/page.tsx` — integrado com Stripe PaymentIntent

### Fluxo de pagamento (resumo)

1. Cliente preenche checkout
2. Criação de pedido `orders` com status `pending`
3. Criação de PaymentIntent no Stripe
4. Pagamento (redirect ou Elements)
5. Webhook atualiza pedido para `paid`
6. Confirmação ao cliente

### Testar localmente

1. Chaves de **teste** e:

   ```
   NEXT_PUBLIC_APP_URL=http://localhost:3000
   ```

2. Terminal 1: `npm run dev`

3. Terminal 2: Stripe CLI — `stripe listen --forward-to localhost:3000/api/stripe/webhook` e usar o `whsec_` mostrado no `.env` como `STRIPE_WEBHOOK_SECRET`; reiniciar o dev server.

4. Testar checkout com cartão de teste; após sucesso, `/checkout/success` e pedido "pago".

### Cartões de teste Stripe

- **Sucesso:** `4242 4242 4242 4242`
- **Falha:** `4000 0000 0000 0002`
- **3D Secure:** `4000 0025 0000 3155`

Data futura e CVC de 3 dígitos.

---

## 4. Build sem Stripe / correções de build

O Next.js/Turbopack pode resolver módulos durante o build mesmo com importação dinâmica.

### Opção 1: Instalar Stripe (recomendado)

```
npm install stripe
```

Sem chaves configuradas, o código pode devolver erros apropriados quando o Stripe não estiver configurado.

### Opção 2: Desativar rotas API temporariamente

1. Renomear `src/app/api/stripe` para `src/app/api/stripe.disabled`
2. Executar build
3. Reverter quando for usar Stripe

### Opção 3: `next.config.ts`

O projeto pode estar configurado para ignorar Stripe no build; o Turbopack pode não respeitar totalmente.

**Nota:** O código pode estar preparado para funcionar sem Stripe configurado (ex.: 503), mas o **pacote** pode ser necessário para o build concluir.

---

## 5. Tabela `order_items` pelo Dashboard Supabase

Se o SQL não fizer a tabela aparecer na API, crie pela interface.

### Table Editor

1. Supabase Dashboard → projeto
2. **Table Editor** → **New table**

### Tabela

- **Name:** `order_items`
- **Schema:** `public`
- **RLS:** ligado

### Colunas

| Name | Type | Nullable | Default |
|---|---|---|---|
| id | uuid | No | `gen_random_uuid()` |

| | | | |
|---|---|---|---|
| order_id | uuid | No | FK → `orders.id` |
| product_id | uuid | No | — |
| quantity | int4 | No | `1` |
| price | numeric | No | — |
| garage_id | uuid | Yes | — |
| created_at | timestamptz | No | `now()` |

`id` como primary key. `order_id` com foreign key para `orders.id` quando possível.

### Policies RLS

**Insert —** `Users can insert order items`

- Operação: **INSERT**
- Roles: `authenticated`
- WITH CHECK:

```
exists (select 1 from public.orders o where o.id = order_items.order_id and o.user_id =
auth.uid())
```

**Select —** `Order items viewable by owner`

- Operação: **SELECT**
- Roles: `authenticated`
- USING: mesma expressão `exists (...)` acima.

### Teste

Após guardar, testar **Place order** na aplicação.

---

## 6. SUPABASE_DB_URL e checkout ( `order_items` )

O checkout insere em **order_items** via API com **conexão direta à base** para contornar "table not found in schema cache" no Supabase.

### Passos

1. **Connection string:** Dashboard → **Settings** → **Database** → **Connection string** (URI / Session mode).
   Formato típico:

   ```
   postgresql://postgres.[PROJECT-REF]:[PASSWORD]@aws-0-
   [REGION].pooler.supabase.com:6543/postgres
   ```

   Substituir a password pela da base de dados do projeto.

2. `.env` **:**

   ```
   SUPABASE_DB_URL=postgresql://postgres.xxxxx:PASSWORD@...
   ```

3. **Reiniciar** `npm run dev`.
4. **Testar checkout** — itens gravados em `order_items` mesmo que a REST API ainda não veja a tabela no schema cache.

### Segurança

- Não fazer commit do `.env` nem partilhar `SUPABASE_DB_URL`.
- A API `/api/checkout/order-items` deve restringir inserções a pedidos do utilizador autenticado.

---

## 7. Script SQL: corrigir API `order_items`

Se a tabela **já existe** mas a API devolve "not found in schema cache", executar no **SQL Editor** do Supabase:

```sql
-- Se a tabela order_items JÁ EXISTE mas a API devolve "not found in schema cache",
-- execute isto no Supabase SQL Editor para dar permissões e recarregar o schema.

grant usage on schema public to anon, authenticated;
grant all on public.order_items to anon, authenticated;
grant all on public.order_items to service_role;
notify pgrst, 'reload schema';
```

(Ficheiro no repositório: `fix_order_items_api.sql`.)

---

## 8. Scripts SQL no repositório (referência)

Lista de ficheiros `.sql` na raiz do projeto (migrações e utilitários). Executar no Supabase conforme a necessidade do ambiente:

- `add_footer_links_and_content.sql`
- `add_garantie_mecanidoc_page.sql`
- `add_garantie_pneus_page.sql`
- `add_modes_livraison_page.sql`
- `add_pa_tipo_column.sql`
- `add_product_category_to_subcategories.sql`
- `add_stripe_fields.sql`
- `create_brands_table.sql`
- `create_category_pages_table.sql`
- `create_faq_table.sql`
- `create_footer_links_table.sql`
- `create_main_category_pages.sql`
- `create_master_user.sql`
- `create_menu_subcategories_table.sql`
- `create_order_items_only.sql`
- `create_order_items_table.sql`
- `create_orders_table.sql`
- `create_promotions_table.sql`
- `create_support_system.sql`
- `create_taxes_table.sql`
- `create_taxes_table_safe.sql`

- `drop_order_items_product_fk.sql`
- `ensure_orders_columns.sql`
- `fix_login_redobrai.sql`
- `fix_order_items_api.sql`
- `link_existing_products_to_brands.sql`
- `run_all_migrations.sql`
- `supabase_schema.sql`
- `update_category_pages_structure.sql`
- `update_existing_products_autres_categories.sql`
- `update_garages_schema.sql`
- `update_master_password.sql`
- `update_modes_livraison_content.sql`
- `update_orders_schema.sql`
- `update_products_brand_link.sql`
- `update_products_schema.sql`
- `update_profiles_schema.sql`

Para uma ordem sugerida de execução, consulte o conteúdo de `run_all_migrations.sql` no repositório.

---

## 9. Scripts npm

Definidos em `package.json`:

| Script | Comando |
|---|---|
| `dev` | `node scripts/run-next.js dev` |
| `dev:clean` | Limpa `.next` e arranca dev |
| `dev:next` | `next dev` |
| `build` | `node scripts/run-next.js build` |
| `start` | `next start` |
| `lint` | `eslint` |
| `stripe:listen` | `stripe listen --forward-to localhost:3000/api/stripe/webhook` |

---

*Fim da documentação consolidada MecaniDoc.*